MSIN0097 – Predictive Analytics Individual Coursework

Churn Prediction Using Telco Customer Data

Candidate Number: SZQS2

Word count : 1985 , Page count : 18

Github repository link : https://github.com/chianyou/msin0097-predictive-analytics

---

1. Introduction and Problem Framing

1.1 Business Context

Customer churn represents a significant challenge in subscription-based industries, where retaining existing customers is generally more cost-effective than acquiring new ones. In telecommunications, customer attrition directly affects revenue stability and long-term profitability. Early identification of customers at risk enables targeted retention strategies, supporting improved customer lifetime value and operational efficiency.

1.2 Dataset and Predictive Task Definition

This study uses the **Telco Customer Churn dataset**, comprising 7,043 customer records and 21 variables describing demographics, subscription characteristics, contract types, billing information, and service usage. The dataset is a static snapshot in which the target variable indicates whether a customer has churned. Its combination of categorical and numerical features makes it suitable for evaluating alternative modeling approaches.

Churn prediction is formulated as a supervised binary classification task, where *Churn* is encoded as 1 (Yes) and 0 (No). The objective is to estimate churn probability and apply a decision threshold to convert predicted probabilities into binary actions. Rather than adopting the default threshold of 0.5, the threshold is treated as a tunable parameter to better align model behaviour with operational priorities.

1.3 Success Metrics and Operational Constraints

Given the moderate class imbalance (approximately 26.5% churn), **PR-AUC** is selected as the primary evaluation metric, as it better reflects minority-class detection performance than overall accuracy. Secondary metrics include Recall, Precision, F1-score, and Balanced Accuracy to characterise trade-offs. Because predicted probabilities may inform prioritisation strategies, calibration is evaluated using the Brier score.

In addition to performance considerations, interpretability was treated as operational requirement because outputs may be used in customer-facing retention actions. Therefore, model choice balanced discrimination performance with transparency.

1.4 Assumptions, Limitations, and Agent Plan

This analysis relies on a static dataset without temporal structure and does not include an explicit economic cost function; threshold optimisation was therefore statistical rather than cost-based.

Agent tooling was used as a structured collaborator for EDA planning, preprocessing scaffolding, and candidate-model prototyping. All metric choices, split discipline, and final model decisions were independently verified through controlled experiments and saved artifacts.

---

2. Exploratory Data Analysis

A concise visual exploratory data analysis (EDA) was conducted to understand churn behaviour, identify data quality issues, and anticipate modelling risks. All findings were validated against generated plots and summary tables from the 01_eda.ipynb notebook.

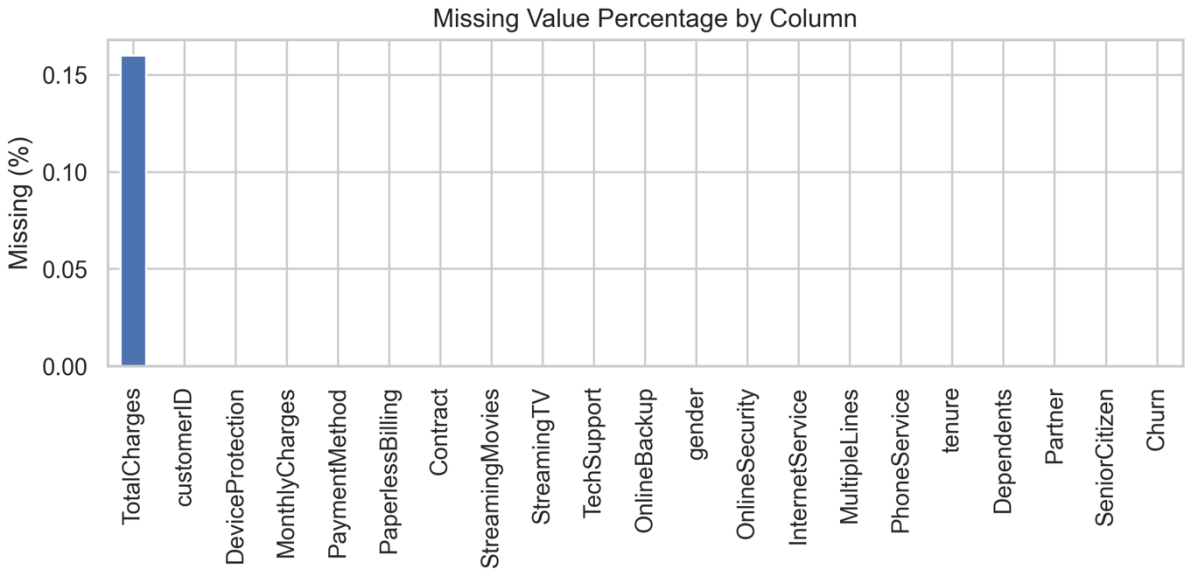
2.1 Dataset Overview

- 7043 rows, 21 columns
- Class imbalance (~26.5% churn)

2.2 Missingness and Data Quality

Missing values are minimal and concentrated in one feature:

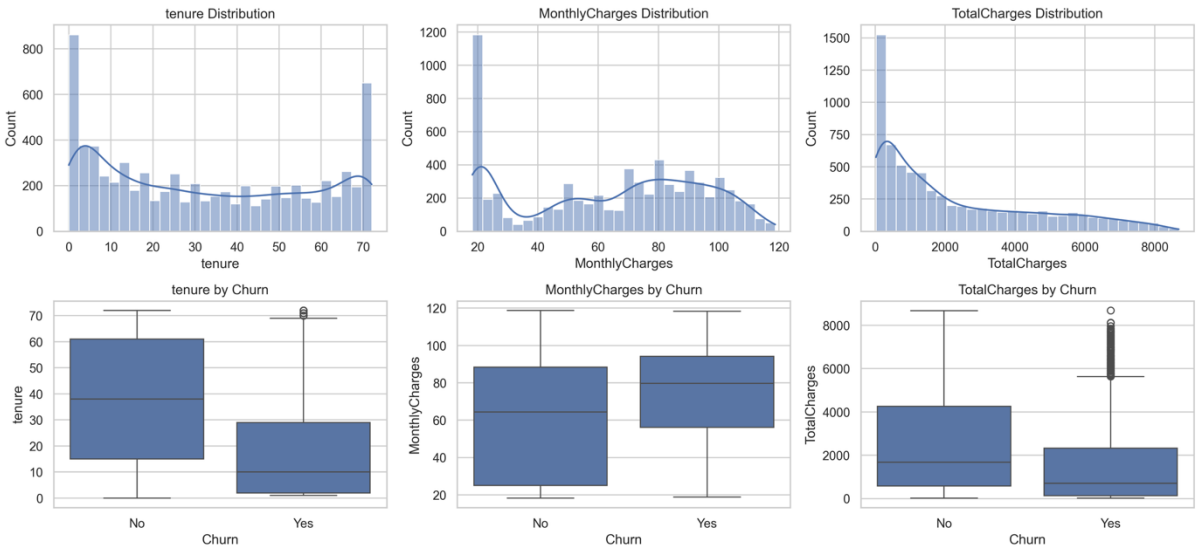
- **TotalCharges:** 11 missing values (0.16%)



▲ (*01_missing_value_percentage_by_column.png*)

This represents low-risk missingness and can be addressed through median imputation without introducing major bias. The issue primarily originated from data type inconsistencies (blank strings converted to NaN), highlighting the importance of preprocessing validation.

2.3 Outlier Assessment(Numerical Features)



▲ *(03_numerical_distributions_by_churn.png)*

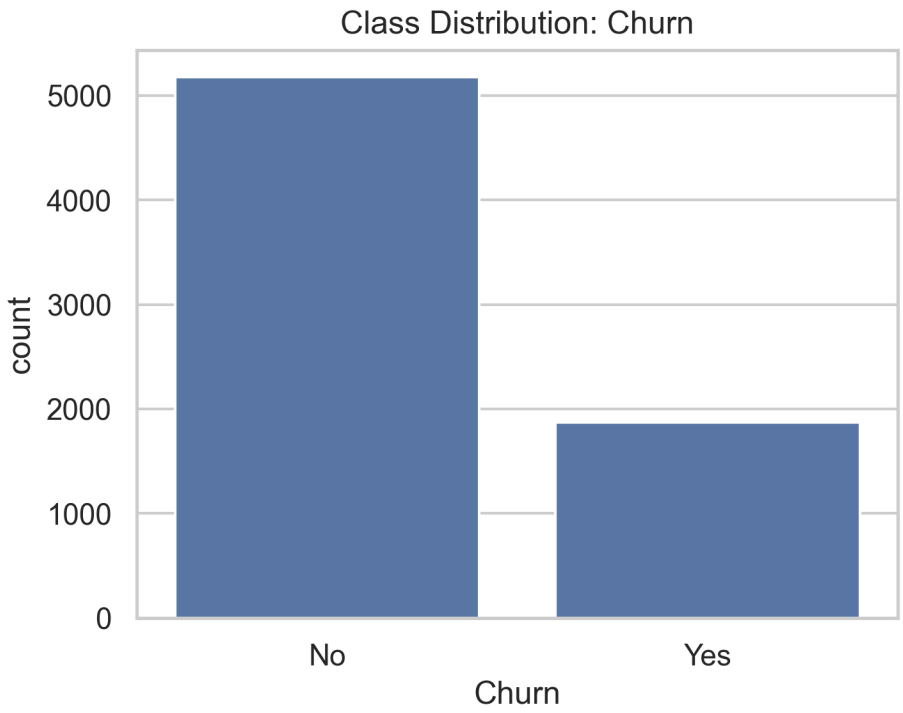
Visual inspection of Figure 03 shows right-tailed distributions and apparent outliers in tenure, MonthlyCharges, and TotalCharges, but these reflect realistic customer behaviour rather than data errors. TotalCharges is most right-skewed due to cumulative billing among long-tenure customers (mostly Churn = No). Higher MonthlyCharges are more common in churned users, while churned customers also exhibit shorter tenure. As these extremes carry

business meaning, outliers were retained, with robustness checks (e.g., winsorisation or log transformation) considered during modelling.

2.4 Class Imbalance and Target Behaviour

The target variable shows a clear imbalance:

- **Churn = Yes:** 26.54%
- **Churn = No:** 73.46%

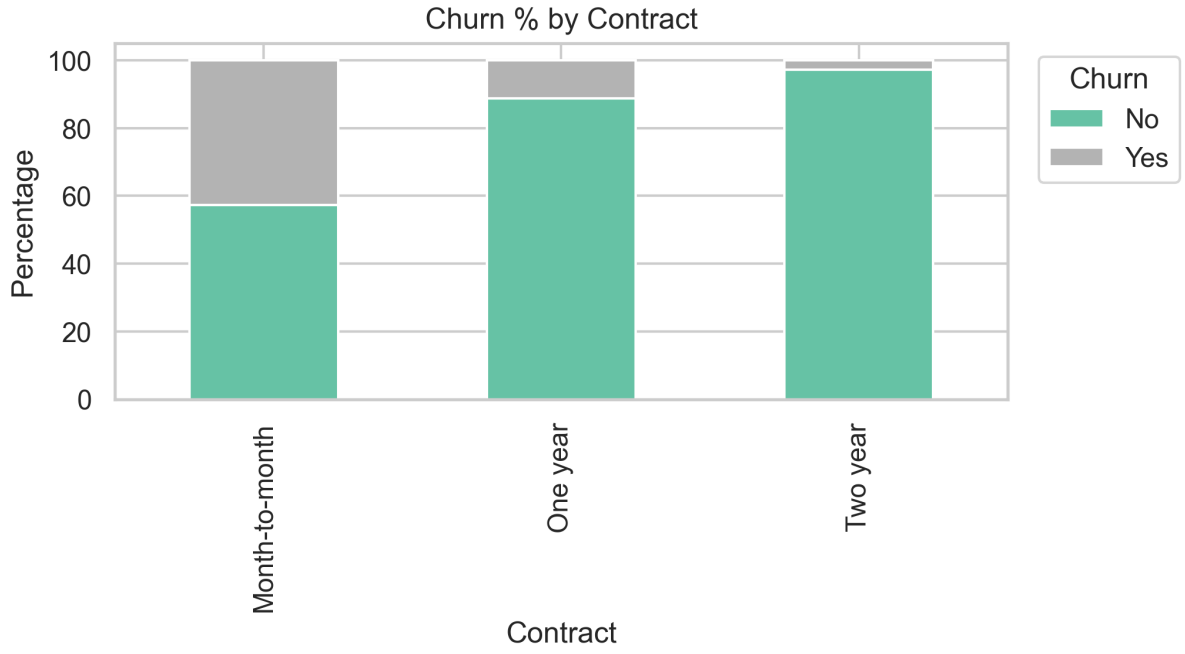


▲ (*02_class_distribution_churn.png*)

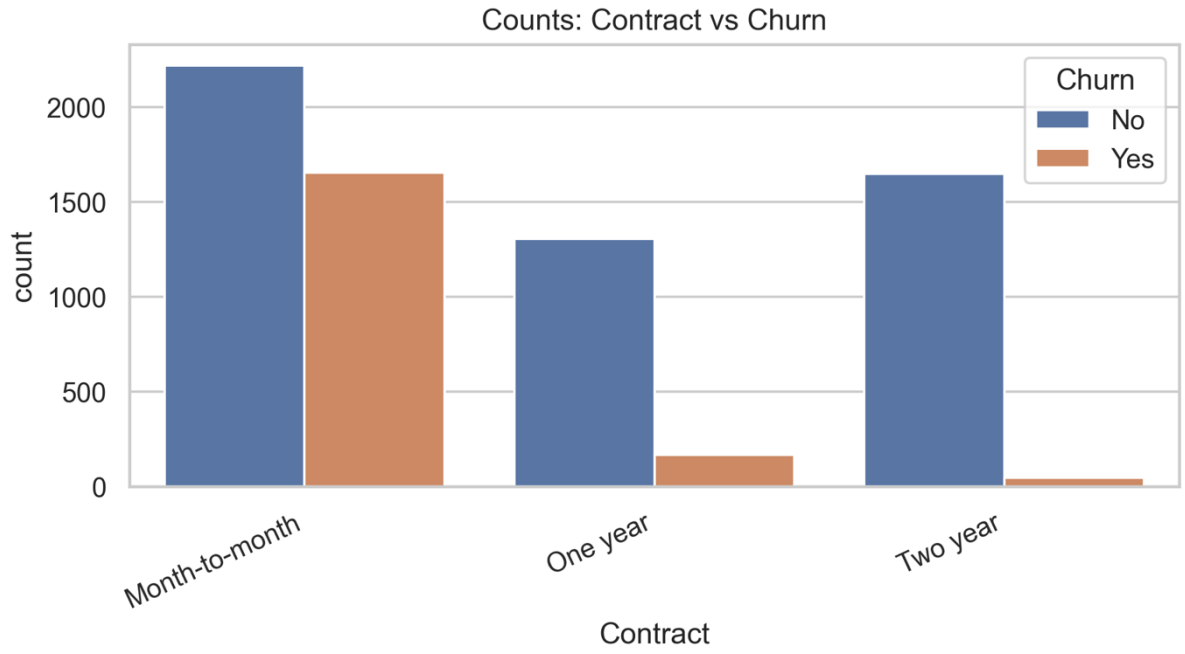
This indicates that accuracy alone would be misleading. Evaluation should prioritise imbalance-aware metrics such as PR-AUC, Recall, and F1-score. Stratified sampling will be required during model training to preserve class proportions.

2.5 Strongest Categorical Drivers of Churn

Contract Type — Primary Risk Factor



▲ (*04_churn_pct_by_contract.png*)

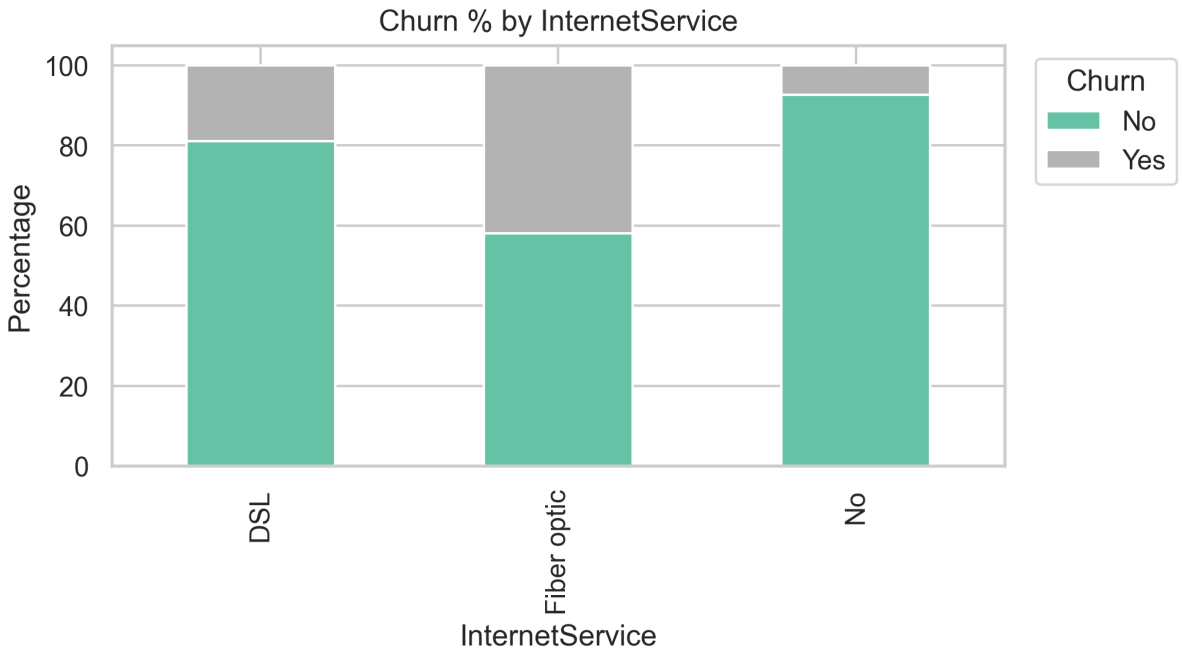


▲ *(05_counts_contract_vs_churn.png)*

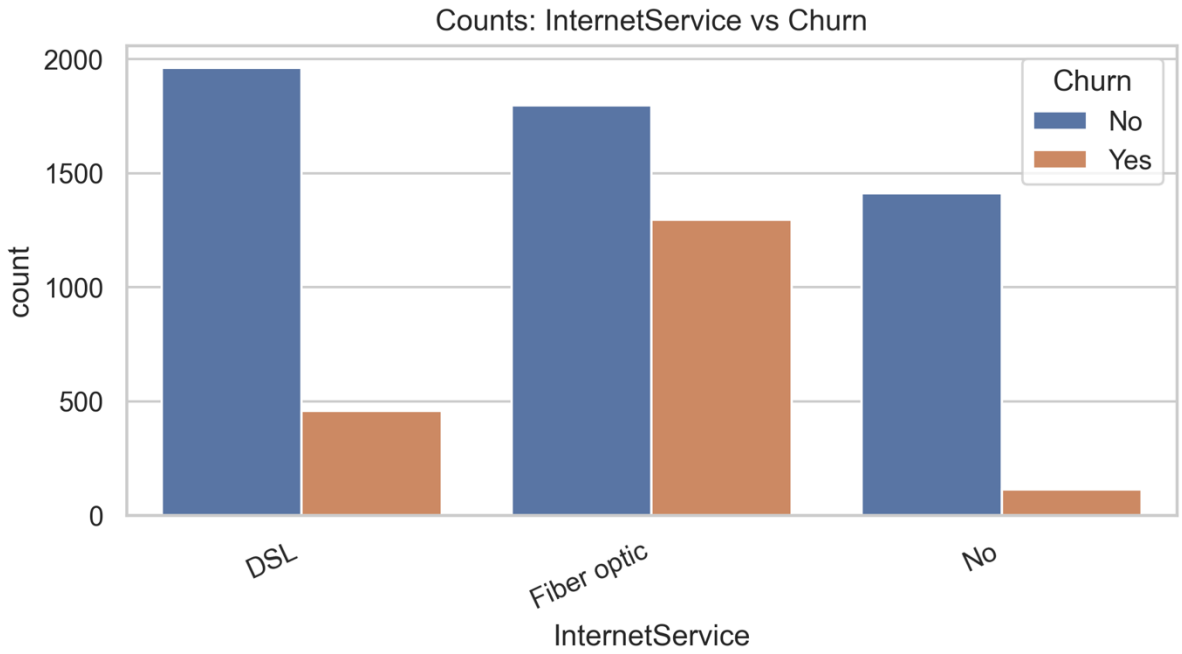
- Month-to-month: **42.71% churn**
- One-year: 11.27%
- Two-year: 2.83%

Contract length demonstrates the strongest separation between churn and non-churn groups, suggesting that customer commitment level is a critical behavioural driver. From a business perspective, short-term contracts represent the highest retention priority.

2.6 Internet Service Differences



▲ (*06_churn_pct_by_internetservice.png*)



▲ *(07_counts_internetservice_vs_churn.png)*

- DSL: 18.96%
- Fiber optic: **41.89% churn**

- No internet: 7.40%

The elevated churn among fiber optic users may reflect pricing sensitivity, service expectations, or support experience

2.7 Leakage Screening

A leakage risk check was conducted by identifying ID-like columns, high-cardinality categorical variables, and near-perfect churn separation patterns. The customerID field was confirmed to function purely as an identifier and was removed during preprocessing. Based on exploratory leakage screening (ID-like columns, cardinality checks, and near-perfect separation patterns), no other feature showed strong leakage signals; however, this should be interpreted as a practical screening result rather than a formal causal leakage proof.

Feature	Unique Count	Notes
customerID	7043	ID-like column (identifier)
PaymentMethod	4	Normal categorical feature
DeviceProtection	3	Normal categorical feature
Contract	3	Normal categorical feature
StreamingMovies	3	Normal categorical feature
StreamingTV	3	Normal categorical feature
TechSupport	3	Normal categorical feature
OnlineBackup	3	Normal categorical feature
OnlineSecurity	3	Normal categorical feature
InternetService	3	Normal categorical feature

▲ *(Table A — ID-like and High-Cardinality Feature Screening)*

Feature	Correlation with Churn_bin	Interpretation
MonthlyCharges	0.193356	Weak positive association with churn
SeniorCitizen	0.150889	Slight positive association
TotalCharges	-0.199484	Negative association (longer customers accumulate charges)
tenure	-0.352229	Strongest negative association with churn

▲ (*Table B — Numerical Correlation with Churn)*

customerID	Churn = No	Churn = Yes	Observation
0002-ORFBO	1.0	0.0	Single-instance separation
0003-MKNFE	1.0	0.0	Single-instance separation
0004-TLHLJ	0.0	1.0	Single-instance separation
0011-IGKFF	0.0	1.0	Single-instance separation
0013-EXCHZ	0.0	1.0	Single-instance separation

▲ *(Table C — Potential Perfect-Separation Signal (Identifier Check))*

3. Data Preparation and Reproducibility

3.1 Reproducible Preprocessing Workflow

A reproducible preprocessing pipeline was implemented in *data_prep.py* using a leakage-safe design and strict train/validation/test discipline.

3.2 Pipeline design and data preparation

All stochastic operations use a fixed random seed (*random_state=42*) to ensure reproducibility. The dataset is split with stratification into train/validation/test (60%/20%/20%). Preprocessing is implemented via a Pipeline and ColumnTransformer, including identifier removal (customerID), TotalCharges numeric coercion, median imputation for numeric features, most-frequent imputation for categorical features, and one-hot encoding with handle_unknown="ignore".

3.3 Leakage control and split discipline

To avoid data leakage, data splitting occurs before any learned transformation. The preprocessor is fitted only on the training set and applied to validation and test sets using transform only. The validation set is used for threshold tuning, while the test set remains strictly held out for final evaluation.

3.4 Data validation and reproducibility checks

Explicit validation steps confirm required columns, consistent target encoding (No→0, Yes→1), split integrity, and transformed output quality (numeric-only features with no missing values). Processed datasets and metadata can be saved to `data/processed/` to enable full reproducibility and auditability.

Validation item	Evidence	Status
Split integrity (`X`/`y` alignment)	Train `(4225,20)/(4225,)`, Val `(1409,20)/(1409,)`, Test `(1409,20)/(1409,)`	Pass
Stratified split consistency	All splits keep nearly identical class ratios: 0:73.46% / 1:26.54%	Pass
Leakage-safe preprocessing	In `prepare_telco_data()`: split first, then `fit_transform(train)` and `transform(val/test)`	Pass
Transformed shape consistency	Train `(4225,45)`, Val `(1409,45)`, Test `(1409,45)`	Pass
Post-transform data quality	`validate_transformed_outputs()`: numeric-only outputs and no NaN after preprocessing	Pass

▲ *(Table D — Data Validation and Pipeline Integrity Checks)*

4. Model Exploration and Selection

Customer churn was formulated as a binary classification task. Multiple candidate models were compared using 5-fold stratified cross-validation on the training split to ensure fair evaluation under class imbalance (~26.5% churn).

Given the imbalance, **PR-AUC (Average Precision)** was used as the primary model-selection metric, with ROC-AUC, F1, recall, precision, and balanced accuracy as secondary indicators.

The evaluated models included:

- Dummy (most frequent) baseline
- Logistic Regression
- Logistic Regression (class-weighted)
- Random Forest
- HistGradientBoosting

Cross-validation results indicated that **Logistic Regression** achieved the highest PR-AUC (0.6667 ± 0.0279), slightly outperforming the weighted variant (0.6654 ± 0.0285), while tree-based models showed lower PR-AUC under this configuration. Considering both predictive performance and interpretability, Logistic Regression was selected as the final model.

To prevent data leakage, hyperparameter tuning was restricted to the training split, threshold optimisation was conducted on the validation split, and the test split was used only once for final reporting. The final model was refitted on train+validation data and evaluated on the test set using the validation-selected threshold.

This protocol ensures fair comparison under class imbalance, transparent threshold control for business trade-offs, and fully reproducible results through a fixed random state (*random_state=42*).

Model	PR-AUC (CV)	ROC-AUC (CV)	F1 (CV)	Recall (CV)	Precision (CV)	Balanced Acc (CV)	Key Interpretation
logreg	0.6667 ± 0.0279	0.8475 ± 0.0119	0.5975 ± 0.0239	0.5540 ± 0.0321	0.6498 ± 0.0302	0.7229 ± 0.0156	Highest PR-AUC and precision; more conservative churn detection

logreg_balanced	0.6654 ± 0.0285	0.8480 ± 0.0130	0.6343 ± 0.0195	0.8073 ± 0.0281	0.5227 ± 0.0206	0.7703 ± 0.0166	Best Recall, F1, and Balanced Accuracy; suitable when retention priority is high
random_forest	0.6533 ± 0.0060	0.8399 ± 0.0111	0.6062 ± 0.0214	0.6272 ± 0.0326	0.5873 ± 0.0224	0.7338 ± 0.0157	Balanced overall performance across metrics
hist_gradient_boosting	0.6262 ± 0.0125	0.8251 ± 0.0131	0.5555 ± 0.0265	0.5157 ± 0.0428	0.6045 ± 0.0259	0.6966 ± 0.0166	Slightly weaker performance than the top three models
dummy_most_frequent	0.2653 ± 0.0005	0.5000 ± 0.0000	0.0000 ± 0.0000	0.0000 ± 0.0000	0.0000 ± 0.0000	0.5000 ± 0.0000	Baseline model for reference comparison

▲ *(Table E —Model Comparison Summary (Cross-Validation Results))*

---

5. Fine-Tuning and Robust Evaluation

5.1 Fine-tuning and Evaluation

A leakage-safe fine-tuning and evaluation workflow was applied. Logistic Regression hyperparameters were tuned using 5-fold stratified cross-validation on the training split only, with PR-AUC as the primary optimization objective. The best configuration was C = 0.1 and class_weight = None, with best CV PR-AUC = 0.6673.

Decision-threshold selection was then performed on the validation split by maximizing F1, yielding an optimal threshold of 0.36. This threshold was fixed before final testing. The final model was refit on train+validation and evaluated once on the held-out test set.

Test Performance

- PR-AUC: 0.6363
- ROC-AUC: 0.8417
- F1-score: 0.6163

- Recall: 0.6872
- Precision: 0.5587
- Balanced Accuracy: 0.7455
- Brier Score: 0.1374

Step 5 Area	Key Result	Value
Hyperparameter tuning (CV)	Best C	0.1
Hyperparameter tuning (CV)	Best class_weight	None
Hyperparameter tuning (CV)	Best CV PR-AUC	0.6673
Threshold tuning (validation)	Selected threshold	0.36
Final test evaluation	Applied threshold	0.36

▲ *(Table F — Hyperparameter Tuning and Threshold Selection Summary)*

Configuration	Mean PR-AUC	Std PR-AUC
C = 0.1, class_weight = None	0.6673	0.0240
C = 1, class_weight = None	0.6667	0.0250
C = 3, class_weight = None	0.6666	0.0250

▲ *(Table G — Top Logistic Regression Configurations Ranked by PR-AUC)*

Scenario	Threshold	F1	Recall	Precision	Balanced Accuracy	Interpretation
Best F1	0.36	0.6334	0.6952	0.5817	0.7573	Best overall trade-off
Best Balanced Accuracy	0.25	0.6226	0.7941	0.5121	0.7603	More recall-oriented
Max Recall	0.05	0.5116	0.9706	0.3474	0.6558	Aggressive churn capture, many false positives
Default baseline point	0.50	0.5884	0.5294	0.6622	0.7159	Higher precision, lower recall

▲ *(Table H1— Validation Threshold)*

5.2 Robust Evaluation and Error Analysis

The test confusion matrix was **[[832, 203], [117, 257]]** (TN, FP, FN, TP).
The model demonstrates relatively strong churn detection (recall = 68.7%), though **117 false negatives** remain, which may be costly in retention-focused applications. At the selected threshold (0.36), the model also produces **203 false positives**, reflecting a precision–recall trade-off consistent with prioritising churn capture.

Probability calibration was assessed using validation and test calibration curves together with the Brier score (0.1374), indicating acceptable probability quality for threshold-based decision-making, while suggesting potential gains from further calibration techniques.

Metric	Value
PR-AUC	0.6363
ROC-AUC	0.8417
F1	0.6163
Recall	0.6872
Precision	0.5587
Balanced Accuracy	0.7455
Brier Score	0.1374
Confusion Matrix [[TN, FP], [FN, TP]]	[[832, 203], [117, 257]]

▲ *(Table H2— Final Test Metrics (Threshold = 0.36))*

---

6. Final Model and Deployment Considerations

6.1 Final Model Selection and Rationale

The final model is **Logistic Regression** (*C = 0.1*, *class_weight = None*) with a decision threshold of **0.36**. It was selected based on the strongest cross-validated PR-AUC performance, stable results across folds, and high interpretability. Compared with more complex ensemble methods, Logistic Regression provides clearer feature effects and easier governance, making it more suitable for business deployment and decision transparency.

6.2 Limitations, Risks, and Next Steps

Several limitations and risks were identified:

- **Class imbalance:** With churn at ~26.5%, evaluation outcomes are sensitive to metric and threshold choice.
- **Error trade-off:** At the selected threshold, the model still produces false negatives (missed churners) and false positives (unnecessary interventions).
- **Data scope:** The dataset represents a static snapshot, creating potential temporal drift risk as pricing, promotions, and customer behaviour evolve.

- **Feature coverage:** Lack of detailed behavioural logs (e.g., service usage intensity or support interactions) may limit achievable recall.
- **Calibration:** Probability estimates are acceptable but could be further improved for cost-sensitive decision-making.

At the selected threshold (0.36), the model prioritizes churn capture over strict precision. Compared with the default threshold (0.50) on validation, recall increases from 0.5294 to 0.6952 (+0.1658), while precision decreases from 0.6622 to 0.5817 (-0.0805). This trade-off is consistent with retention-focused operations where missing true churners is more costly than contacting some non-churners.

Next steps:

- Implement cost-sensitive thresholding using explicit business costs for FP/FN trade-offs.
- Conduct temporal validation (if timestamped data becomes available) to assess drift robustness.
- Enrich feature space with behavioural or interaction variables and evaluate incremental uplift.
- Apply post-hoc calibration methods (e.g., isotonic regression or Platt scaling) and reassess Brier score and calibration curves.
- Define operational risk bands rather than relying on a single hard decision threshold.

6.3 Model Card (Summary)

Intended Use

- **For:** Prioritising telecom customers by churn risk to support retention strategies.
- **Not for:** Automated approval/denial decisions, pricing discrimination, or deployment outside comparable telecom contexts without revalidation.

Data Provenance and Constraints

- **Source:** Kaggle Telco Customer Churn dataset (7,043 rows, 21 columns).
- **Target:** Churn (Yes/No), encoded as 1/0.
- **Preprocessing:** Identifier removal, `TotalCharges` cleaning, median/mode imputation, and one-hot encoding.
- **Constraints:** Static tabular snapshot with limited behavioural depth and potential temporal or sampling bias.

Evaluation Summary

- **Selection metric:** PR-AUC via 5-fold stratified CV on training data.
- **Best CV PR-AUC:** 0.6673.
- **Test performance:** PR-AUC 0.6363, ROC-AUC 0.8417, F1 0.6163, Recall 0.6872, Precision 0.5587, Balanced Accuracy 0.7455, Brier Score 0.1374.
- **Confusion matrix:** [[832, 203], [117, 257]] (TN, FP, FN, TP).

Caveat: Model performance is threshold-dependent; real-world deployment should adjust thresholds based on intervention capacity and business cost structure.

Appendix

- **Decision Register table**

Project Stage	Agent Suggestion / Output	Your Verification Process	Decision	Rationale / Outcome
EDA – Class Imbalance	Suggested inspecting churn distribution	Verified counts via class distribution plot	Accepted	Confirmed imbalance; informed metric selection
EDA – Leakage Screening	Proposed ID-like and high-cardinality checks	Ran separation tests; confirmed customerID be haves as identifier	Modified	Reframed as leakage screen ing, not actual leakage feature
EDA – Outlier Handling	Suggested possible transformations	Checked distributions and business logic	Modified	Outliers retained; represent real billing behaviour
Feature Engineering	Recommend One-Hot Encoding with unknown handling	Validated categories across splits	Accepted	Prevents inference-time failure
Data Split Strategy	Use stratified 60/20/20 split with fixed seed	Verified class balance across splits	Accepted	Ensures reproducibility and fair evaluation
Preprocessin g Pipeline	Pipeline + ColumnTransfor mer design	Checked fit only on train split	Accepted	Leakage-safe workflow confirmed
Metric Selection	Use PR-AUC as primary metric	Checked imbalance context	Refined	PR-AUC kept primary; added secondary metrics for robustness

Hyperparameter Tuning	Tune C parameter via CV	Verified tuning restricted to train split	Accepted	Prevents validation leakage
Threshold Optimisation	Select threshold via validation F1	Analysed recall–precision trade-off	Modified	Threshold chosen based on business recall priority
Calibration Check	Evaluate Brier score and calibration curves	Inspected probability reliability	Accepted	Probability outputs usable but improvable
Error Analysis	Suggest confusion matrix review	Verified FN/FP trade-off	Accepted	Supports retention-focused interpretation

▲ *(Table I — Agent Contribution and Verification Decision Register)*

- **Agent Usage Log screenshots**

don't change my code Help me design a scikit-learn Pipeline with ColumnTransformer for tabular churn data.
Numeric features need median imputation, categorical features need most frequent imputation plus one-hot encoding.
Make sure the pipeline is safe against data leakage.

Use this pattern:

python

```python
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression  # example model

# Example: X = df.drop(columns=["Churn"]), y = df["Churn"]
# Ensure y is encoded (e.g., Yes/No -> 1/0) if needed.

numeric_features = ["tenure", "MonthlyCharges", "TotalCharges"]   # replace with your numeri
categorical_features = ["Contract", "PaymentMethod", "InternetService"]  # replace with your

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median"))
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, numeric_features),
        ("cat", categorical_pipeline, categorical_features),
    ],
    remainder="drop"
)

clf = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", LogisticRegression(max_iter=1000, class_weight="balanced"))
])

# Split BEFORE fitting anything to avoid leakage
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Fit only on training data
clf.fit(X_train, y_train)
```

▲ *(Screenshots 1—Preprocessing Pipeline)*

do not change my code
For a churn dataset with class imbalance (~26%), should PR-AUC be used as the primary metric?
How should I combine it with ROC-AUC, F1, recall, and precision?

Yes, with ~26% churn, **PR-AUC should usually be the primary ranking metric** if your focus is identifying churners (positive class performance).

Use metrics like this:

1. **Primary: PR-AUC**
   - Best for imbalanced settings and reflects performance on churn class directly.
   - Good for model selection/tuning when positives are the business focus.
2. **Secondary: ROC-AUC**
   - Keep as a stability/discrimination check across thresholds.
   - Useful for comparing overall separability, but can look optimistic under imbalance.
3. **Threshold metrics for operating point: Precision, Recall, F1**

- Pick a decision threshold based on business cost:
  - If missing churners is costly: optimize for **high recall** with acceptable precision.
  - If retention actions are expensive: optimize for **high precision**.
- Use **F1** (or better, **F_beta**) to summarize precision-recall tradeoff at chosen threshold.

Recommended workflow:

1. Tune models using cross-validated **PR-AUC** (primary), monitor **ROC-AUC** (secondary).
2. On validation set, choose threshold via precision-recall curve and business constraints.
3. Report final: **PR-AUC, ROC-AUC, Precision, Recall, F1**, plus confusion matrix at chosen threshold.

If you have a strict retention budget, also report **Precision@K / Recall@K** (top-N customers targeted), which is often more actionable than a default 0.5 threshold.

▲ *(Screenshots 2— Metric Selection – PR-AUC)*

How can I tune Logistic Regression hyperparameters (especially C) using stratified cross-validation while avoiding validation leakage?
Show best practices for scikit-learn workflows.

Use `GridSearchCV` (or `RandomizedSearchCV`) on a **full Pipeline** so every fold learns preprocessing only from that fold's training split.

python

```python
import numpy as np
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

# Example feature lists
num_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
cat_cols = ["Contract", "PaymentMethod", "InternetService"]

# Preprocessing inside pipeline (leakage-safe)
num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())  # usually helps LR
])

cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("ohe", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("cat", cat_pipe, cat_cols)
])
```

▲ *(Screenshots 3— Hyperparameter Tuning)*

I want to choose a decision threshold based on validation data rather than default 0.5. How can I optimise threshold using F1 or recall–precision trade-offs for churn retention scenarios?

Use predicted probabilities on a **validation set** and pick the threshold that matches your business objective.

python

```python
import numpy as np
from sklearn.metrics import precision_recall_curve, f1_score, classification_report, confusi

# 1) Fit model on train only
clf.fit(X_train, y_train)

# 2) Get validation probabilities for positive class (churn=1)
val_proba = clf.predict_proba(X_val)[:, 1]

# 3) Build PR curve
precision, recall, thresholds = precision_recall_curve(y_val, val_proba)
# precision/recall have len = len(thresholds)+1, so align:
p, r = precision[:-1], recall[:-1]

# Option A: Max F1 threshold
f1 = 2 * (p * r) / (p + r + 1e-12)
best_idx = np.argmax(f1)
best_thr_f1 = thresholds[best_idx]

# Option B: Recall-first (e.g., recall >= 0.80, maximize precision)
target_recall = 0.80
candidates = np.where(r >= target_recall)[0]
best_thr_recall = thresholds[candidates[np.argmax(p[candidates])]] if len(candidates) else 0

# Option C: Precision-first (e.g., precision >= 0.60, maximize recall)
target_precision = 0.60
candidates = np.where(p >= target_precision)[0]
best_thr_precision = thresholds[candidates[np.argmax(r[candidates])]] if len(candidates) els
```

▲ *(Screenshots 4— Threshold Optimisation)*